24 DE ABRIL DEL 2026

EVALUACIÓN Y SIMULACIÓN DE SENSIBILIDAD A HIPERPARAMETROS

INTEGRANTES:

BRYAN ISMAEL ALVARADO QUIMIZ

DIEGO ANDRÉ ARÁMBULO VÍTORES

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

Definición del caso de estudio

Problema

El presente caso de estudio aborda la problemática en la interpretación de consultas realizadas en lenguaje natural en los dominios de un sistema de gestión documental, en donde los usuarios del mismo formularán las solicitudes haciendo uso de diferentes variaciones lingüísticas tales como: formalidad, ruido, informalidad o contaminación sonora desde sistemas de voz a texto - ASR.

El problema central se enfoca en la variabilidad y la representación de ruido en múltiples formas (errores tipográficos, elongaciones, transcripciones ASR) que afectan la correcta identificación de la real intención del usuario y extracción de entidades relevantes. Todo lo anterior mencionado tiene un impacto directo en las operaciones que el usuario busca ejecutar, podemos mencionar las siguientes:

- Recuperación correcta de documentos
- Experiencia del usuario
- Navegación y guía en el sistema para procedimientos

Por lo tanto, el desafío para solucionar la problemática expuesta consiste en diseñar, evaluar e implementar una solución híbrida robusta capaz de identificar las intenciones correctas del usuario pre procesando correctamente la información recibida, y extraer las entidades en condiciones de ruido y variabilidad lingüística, tanto de parte del texto, como del input obtenido del módulo de voz a texto ASR.

Propuesta de modelo

Debido a las características de la situación en la cual se implementará la solución, se usará una arquitectura híbrida para el procesamiento de lenguaje natural PLN, que se compondrá de 3 componentes especializados y complementarios.

1.- Clasificaciones de intenciones (Zero-Shot Learning)

Como componente de clasificación se optó por el modelo *vicgalle/xlm-roberta-large-xnli-anli*, el cual está basado en *xlm-roberta-large*, pero tiene 2 grandes fine tuning, basados en el dataset XNLI un corpus de inferencia en lenguaje natural, lo que le permite razonar sobre relaciones lógicas en múltiples idiomas, y por otro lado ANLI el cual mediante ejemplos adversariales está diseñado para engañar a los modelos mediante 3 rondas de dificultad. Por sus cualidades desencriptar su enfoque será el de generar inferencia basada en Natural Language Inference (NLI).

El modelo base fue pre entrenado sobre cien idiomas con más de dos terabytes de datos, dotándolo de una sólida capacidad de transferencia entre lenguas (Conneau et al., 2020). Sus fine tunings sobre XNLI, corpus multilingüe de relaciones semánticas en 15 idiomas (Conneau et al., 2018), y sobre ANLI, dataset adversarial de dificultad progresiva en tres rondas (Nie et al., 2020), lo habilitan para clasificación zero-shot robusta y multilingüe.

Uno de los puntos más fuertes es que este componente permite clasificar consultas sin necesidad de entrenamiento supervisado específico del dominio en donde se implementara. Siendo uno de sus objetivos la identificaciones oportuna de las intenciones objetivo tales como:

- Navegación
- Descarga de documentos
- Listar documentos

2.- Análisis lingüístico (estructura y verbos)

Para la manipulación de la estructura del texto input se implementó la herramienta *spaCY*, con el modelo *es_core_news_lg* , cuya cualidad podemos rescatar que es un corpus de los más grandes de la familia es_core_news, podemos inferir que fue especializado para el idioma español, es un pipeline de propósito general entrenado sobre texto de noticias, de gran tamaño con una tabla de vectores completa, posicionando como el modelo en español más completo para uso con cpu. Dentro de la solución, tomará las principales funciones enumeradas a continuación

- Identificación de verbos de acción (token etiquetados)
- Análisis sintáctico (árbol dependencia sintáctica entre componentes)
- Soporte para interpretación contextual (significado interpretado de cada token)

Todos los componentes de spaCY proveen un pipeline de procesamiento robusto ideal para las necesidades de la solución, ya que se adapta a los escenarios reales a los cuales será enfrentado el sistema propuesto. Según Honnibal et al. (2020), spaCY está diseñado para procesamiento de texto a escala industrial, combinando etiquetado morfológico, análisis de dependencias sintácticas y vectores contextuales en un pipeline modular, lo que lo hace especialmente adecuado para entornos de producción que requieren precisión lingüística robusta.

3.- Reconocimiento de entidades (NER)

Dentro de la fase final del módulo tendremos al componente encargado de identificar las diferentes tipos de entidades que los inputs de usuarios pueden contener, esto es de vital importancia para los pasos posteriores cuando se necesite articular y armar la interacción contra el API de gestión documental, la cual solicitará datos y contexto de los mismos. El modelo encargado de esto es *Babelscape/wikineural-multilingual-ner*, basado en arquitectura transformer que produce anotaciones de alta calidad, sin embargo al estar entrenado sobre texto derivado de wikipedia, puede presentar generalizaciones no óptimas, sin embargo se sigue destacando antes modelos entrenados solo sobre noticias cuando es de analizar artículos de carácter enciclopédico.

Para este caso, aplicaremos una estrategia de agregación simple, para que las palabras no sean tokenizadas a nivel de subpalabra, sino que se agrupen en un uno solo por entidad, esta es la opcion mas practica en ambientes productivas ya que no producen sobrecarga computacional adicional (Wolf et al., 2020).

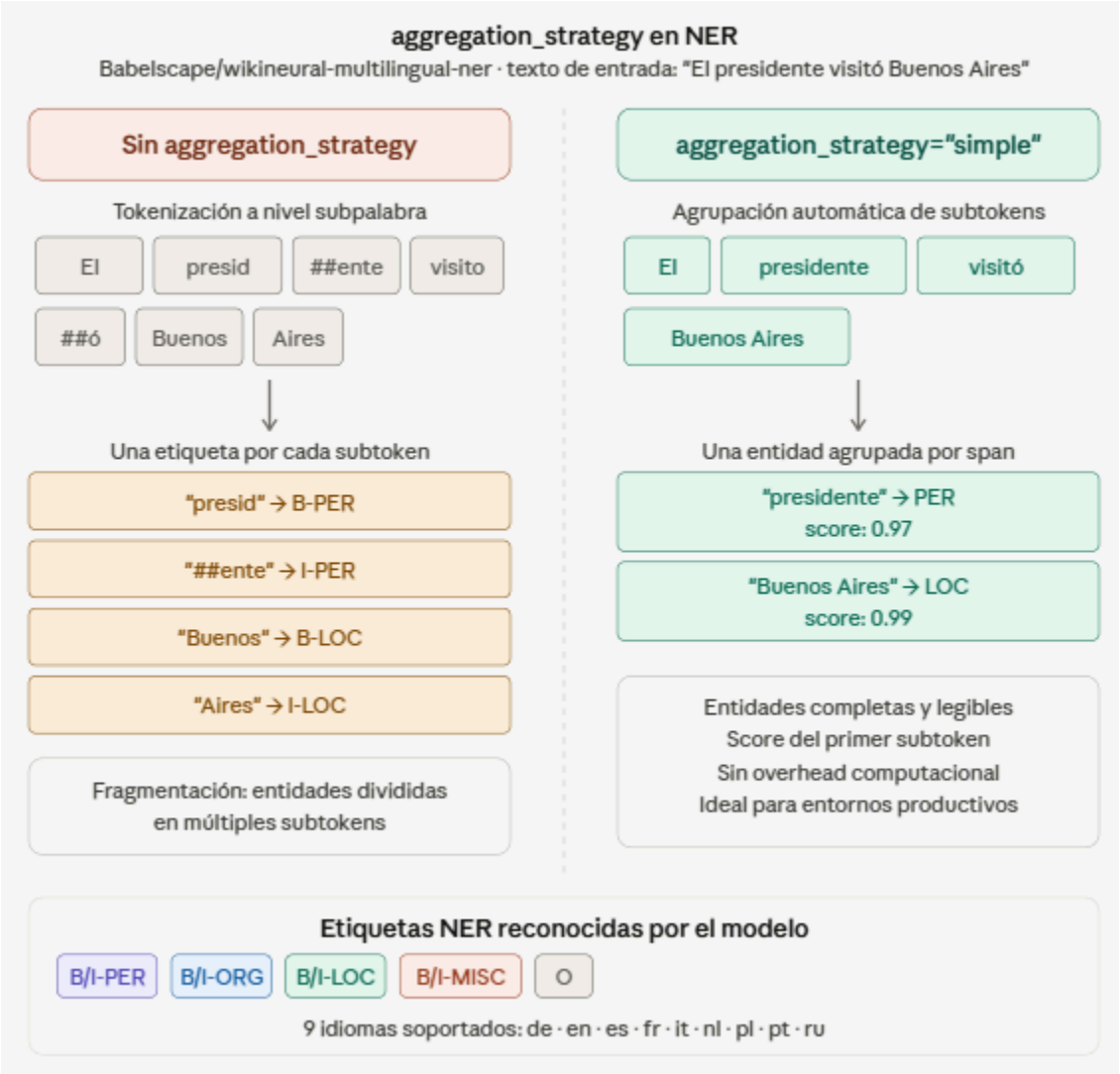


Figura 1: *Comparativa de overhead sin utilizar y con aggregation strategy*

Nota. Autoría propia

Métricas de evaluación

Para medir y evaluar el desempeño del modelo deben tomarse en cuenta aquellas métricas específicamente usadas en otros trabajos y literatura que hayan usado soluciones PLN, entre las estándares podemos mencionar:

- Accuracy
    - Proporción de predicciones correctas sobre el total de inputs obtenidos
- Precision
    - Medición de predicciones positivas correctas
- Recall
    - capacidad de identificación correcta de todas las instancias reales
- F1-score
    - Equilibrio del modelo en base a la armonía de precision y recall
- Fairness
    - tipo de lenguaje (formal vs informal)
    - presencia de ruido (ASR, texto sucio)
    - longitud de la consulta

Las métricas de accuracy, precision, recall y F1-score constituyen el estándar para evaluar clasificadores en PLN (Delgado & Tibau, 2024), mientras que la fairness complementa esta evaluación considerando variaciones lingüísticas, ruido y longitud de entrada (Gallegos et al., 2024).

Hiperparametros relevantes

Un hiper parámetro es un valor de configuración generalmente establecido antes del entrenamiento, y tiene la cualidad que no se aprende de los datos, sino que son ajustes que influyen en las decisiones del modelo. (Claesen & De Moor, 2015). Para nuestro caso que emplea el uso de modelos pre entrenados, estos hiperparametros son considerados de despliegue o inferencia, a continuación revisaremos los más relevantes identificados por componente:

1.- candidate_labels

- **Componente**: Zero-shot intent
- **Valor sugerido**: [“navegación”, “descarga de documento”, “listar documento”]
- **Justificación técnica**: define explícitamente sobre las clases semánticas el espacio sobre el cual el modelo hace inferencia mediante NLI, en modelos basados en XLM-RoBERTa , la clasificación se basa en evaluar la hipótesis contra el texto de entrada (Yin et al., 2019).

2.- multi_label

- **Componente**: Zero-shot intent
- **Valor sugerido**: False
- **Justificación técnica**: define si la clasificación permite múltiples etiquetas de forma simultanea o una frase dominante, ya que variantes de BERT han sido usados en clasificaciones de clase única, ya que como indica (Chen et al., 2019) la clasificación de intenciones es típicamente modelada como una tarea de clasificación de una sola etiqueta.

3.- aggregation_strategy

- **Componente**: NER (Transformers)
- **Valor sugerido**: “simple”
- **Justificación técnica**: Permite agrupar tokens consecutivos en entidades completas, evitando fragmentación (ej. “Juan Ramirez” como una sola entidad PER). Mejora la calidad semántica del output del NER, ya que de no hacerlo así tal como menciona (Tedeschi et al., 2021) La tokenización de subpalabras requiere un post procesamiento para reconstruir las entidades completas.

4.- batch_size

- **Componente**: spaCY (nlp.pipe)
- **Valor sugerido**: 32 (ajustable)
- **Justificación técnica**: Controla el número de textos procesados de forma simultánea en pipelines basados en NLP, no impacta en la precisión del modelo, pero sí tiene relevancia en el rendimiento como es el tiempo, uso de memoria y escalabilidad, paralelizando tareas y usando de forma eficiente el hardware. El procesamiento por minilotes mejora la eficiencia computacional y el comportamiento de convergencia (Goodfellow et al., 2016) .

Diseño y ejecución de experimentos

Para evaluar el comportamiento del módulo de identificación de intenciones, se diseñó una matriz experimental basada en los hiperparametros más relevantes del pipeline híbrido PLN:

- candidate_labels
- multi_label
- aggregation_strategy
- batch_size

El objetivo es analizar cómo diferentes configuraciones afectan la clasificación de intenciones, la extracción de entidades y la eficiencia del procedimiento.

Hiperparámetro	Valores evaluados	Componente
`candidate_labels` Etiquetas de clasificación	3 etiquetas base / 5 etiquetas extendidas	Zero-shot
`multi_label` Etiquetas múltiples	`False` , `True`	Zero-shot
`aggregation_strategy` Agrupación de tokens	`"simple"` , `"first"`, `"average"`	NER
`batch_size` Tamaño de lote	`8, 16, 32, 64`	spaCy

Figura 2: *Hiperparametros evaluados y sus posibles valores*

Nota. Autoría propia

Espacio de búsqueda experimental

La estructura de búsqueda será de tipo estructurada tipo g*rid search controlado*, de la cual toma los hiperparametros y combina los diferentes ajustes permutandolos. No se realizaron combinaciones arbitrarias, sino configuraciones coherentes con el funcionamiento del sistema propuesto

La exploración controlada del espacio de hiper parámetros permite evaluar configuraciones relevantes de manera sistemática, siendo adecuada cuando el número de parámetros es reducido y bien definido (Bergstra & Bengio, 2012).

Exp.	candidate_labels	multi_label	aggregation_strategy	batch_size
E1	Base	False	simple	8
E2	Base	False	simple	16
E3	Base	False	simple	32
E4	Base	False	simple	64
E5	Base	True	simple	32
E6	Extendida	False	simple	32
E7	Base	False	first	32
E8	Base	False	average	32
E9	Extendida	True	simple	32
E10	Extendida	False	average	64

Figura 3: *Tabla de experimentos combinando hiper parámetros*

Nota. Autoría propia

Aquí cabe recalcar que la configuración de experimento E3 es considerada como el punto base del sistema, debido a que emplea los mínimos valores estándares esperados por defecto y en el caso de las etiquetas de intenciones, las que están claramente identificadas sin agregaciones adicionales ya que representan el comportamiento del modelo en condiciones normales. A partir de ese punto se modifican los hiper parámetros de forma controlada.

- **candidate_labels:** se amplía para evaluar si un número mayor de etiquetas mejora o genera ruido a la clasificación
- **multi_label:** su cambio permite comprobar si una consulta se beneficia de múltiples intenciones
- **aggregation_strategy**: se cambia para medir el impacto en la calidad de entidades encontradas
- **batch_size:** se ajusta para medir la eficiencia computacional

Exp.	Accuracy	Precision	Recall	F1-score	Tiempo (s)
E1	0.70	0.2917	0.3333	0.3111	3.9476
E2	0.70	0.2917	0.3333	0.3111	1.9163
E3	**0.70**	**0.2917**	**0.3333**	**0.3111**	**1.9008**
E4	0.70	0.2917	0.3333	0.3111	2.2272
E5	0.70	0.2917	0.3333	0.3111	1.8666
E6	0.60	0.3500	0.3929	0.3247	4.9384
E7	0.70	0.2917	0.3333	0.3111	2.0212
E8	0.70	0.2917	0.3333	0.3111	2.1192
E9	0.50	0.3000	0.2143	0.2455	3.9195
E10	0.60	0.3500	0.3929	0.3247	4.0255

Figura 4: *Tabla de resultados en los experimentos propuestos, resaltando el base esperado*

Nota. Autoría propia

Observación de resultados obtenidos

El diseño experimental se estructuró mediante una búsqueda controlada sobre los hiper parámetros principales del pipeline híbrido de PLN. Se evaluaron combinaciones representativas cubriendo tanto configuraciones conservadoras como configuraciones extendidas. La estrategia permitió analizar el impacto de cada hiper parámetro sobre la clasificación de intenciones, la extracción de entidades y la eficiencia computacional del sistema.

Cada experimento fue registrado con métricas de desempeño como accuracy, precision, recall y F1-score, además del tiempo de ejecución y observaciones cualitativas sobre errores de clasificación o fragmentación de entidades. Esta organización permitió comparar los resultados de forma trazable, reproducible y alineada con los objetivos del Módulo 2.

Insights detectados

- **candidate_labels**: aquellos que usan las etiquetas base mantienen un accuracy del 70%, sin embargo con las extendidas este valor cae al %50, lo que deja claro que a mayor etiquetas, aumenta la confusión del clasificador.
- **multi_label**: no tiene mayor relevancia en el corpus de prueba de 10 registros, y tiene un efecto contrario en la confianza del modelo al tener un valor de True.
- **aggregation_strategy**: el parámetro no cambio el rendimiento en la detección de intenciones, sin embargo en tiempos la etiqueta "simple" demostró ser la más eficiente y suficiente para modelos NER standar
- **batch_size**: este parámetro reduce de forma considerable el tiempo de ejecución, mejorando la eficiencia computacional de inferencia, sin embargo su punto de equilibrio se encuentra entre 16 o 32, a mayo numero es imperceptible su mejora

Análisis de sensibilidad individual

Habiendo tenido listo la comparativa general de hiperparametros, nos centraremos ahora en la sensibilidad individual que cada variable puede generar al ser modificada de forma individual, para eso tomaremos como base el experimento E3 que es el que tiene los parámetros normalizados en el modelo y recrearemos ciertos escenarios.

Sensibilidad de batch_size

Tomando como columna pivote, todos los valores de batch_size, re ejecutaremos las pruebas anteriores para visualizar cómo cambia la métrica más relevante hacia el análisis.

batch_size	F1	Tiempo (s)
8	0.3111	2.2405
16	0.3111	1.8613
32	**0.3111**	**1.9678**
64	0.3111	1.8562

Figura 5: *Rendimiento del modelo ajustando hiper parámetro batch_size por tiempo contra el escenario base*

Nota. Autoría propia

Notamos que el F1 score se mantiene constante, pero el tiempo se reduce al pasar de 8 a 16/32, estabilizando o saturando después de esto. Podemos concluir que el rango óptimo va de 16 a 32 para este hiper parámetro, y la saturación se hace evidente al tener un valor de 64.

Sensibilidad de candidate_labels

El siguiente análisis se llevará a cabo con las etiquetas o grupo de intenciones que el modelo podrá tener, separadas en conjunto base y extendido, se obtuvieron los siguientes valores:

etiquetas	F1
Base	0.3111
Extendida	**0.3247**

Figura 6: *Rendimiento del modelo ajustando hiper parámetro candidate_labels*

Nota. Autoría propia

Al iterar sobre los valores de parámetro vemos una ligera variación en el F1, con las etiquetas extendidas, 0.1 de diferencia

Sensibilidad de multi_label

El hiper parámetro de multi label, nos permite modificar la forma en que el modelo asocia la idea de que múltiples intenciones pueden coexistir, reduciendo la competitividad del softmax, al ser un valor booleano, solo obtendremos 2 conjuntos de evaluación.

El hiper parámetro multi_label nos permite modificar la forma en que el modelo asocia la idea de que múltiples intenciones pueden coexistir, reduciendo la competitividad del softmax. Cuando multi_label = False, los scores se normalizan de modo que la suma de probabilidades de todas las etiquetas candidatas sea 1; cuando multi_label = True, las etiquetas se tratan como independientes entre sí y las probabilidades se normalizan individualmente mediante softmax por cada candidato (Wolf et al., 2020). Este comportamiento es especialmente relevante en escenarios donde coexisten múltiples intenciones en un mismo enunciado, ya que la clasificación zero-shot basada en entailment permite asociar etiquetas a un texto independientemente del dominio y del aspecto descrito por la etiqueta (Yin et al., 2019).

multi_label	F1
False	0.3111
True	0.3111

Figura 6: *Rendimiento del modelo ajustando hiper parámetro multi_label de coexistencia para intenciones detectadas*

Nota. Autoría propia

La métrica se mantuvo sin cambios aparentes podemos evidenciar que no tiene mayor impacto en corpus no extensos como los son las peticiones del usuario.

Sensibilidad de aggregation_strategy

Por último tenemos la evaluación del hiper parámetro con respecto a la estrategia de agregación, la cual nos permite controlar la estrategia de transformers para separar las palabras y generar subtokens, esto tiene como resultado la división de una palabra en varias secciones y un costo computacional posterior al tenerla que reensamblar (Sennrich et al., 2016).

strategy	Calidad NER
simple	**mejor (entidades coherentes)**
first	fragmentado (depende del primer token)
average	inconsistente (promedios confunden)

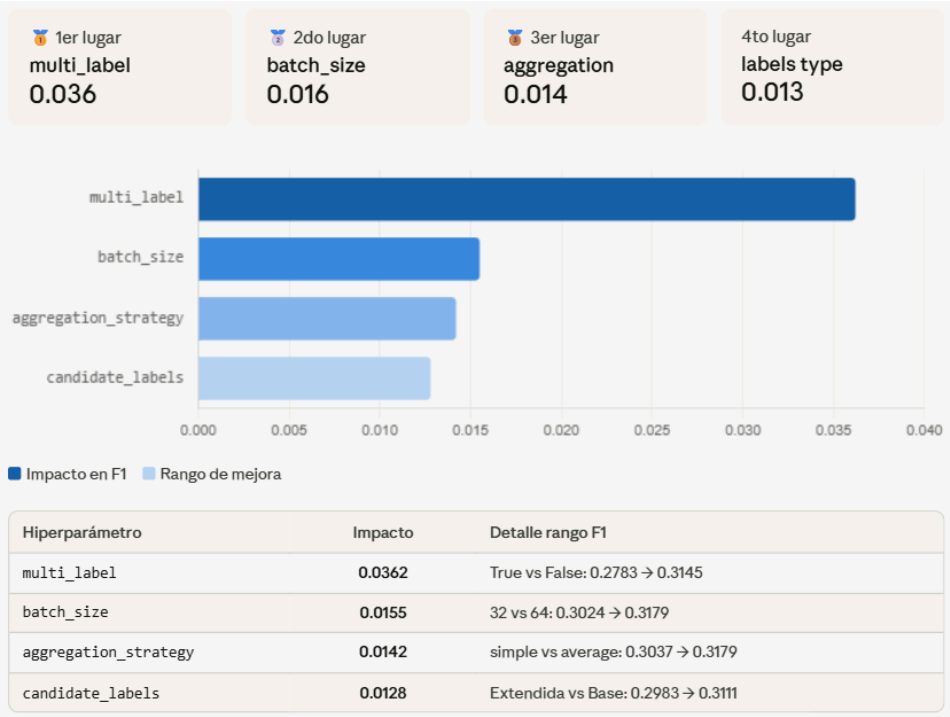
Figura 7: *Rendimiento del modelo ajustando hiper parámetro aggregation_strategy*

Nota. Autoría propia

La comparativa en este indicado es de forma cualitativa, ya que las entidades podrían generarse de 2 formas distintas, unificadas coherentes y fragmentadas e inconsistentes, siendo la más óptima la estrategia simple, al poder reconstruir de una mejor manera las entidades, a diferencia de first o average que descompon mucho la entidad en casos de nombres compuestos o fechas.

Ranking de importancia de hiper parámetros

Para determinar de manera efectiva qué hiper parámetro tiene la mayor importancia al momento de modificarlo, fue implementada la importancia por impacto promedio ANOVA simplificado / efecto marginal, debido a que la idea principal radica en medir la variación de F1-score cuando cada hiper parámetro tiene un cambio. El marco funcional ANOVA permite cuantificar la importancia de hiper parámetros individuales midiendo su efecto marginal promedio sobre el rendimiento del modelo (Hutter et al., 2014).



Hiperparámetro	Impacto	Detalle rango F1
multi_label	0.0362	True vs False: 0.2783 → 0.3145
batch_size	0.0155	32 vs 64: 0.3024 → 0.3179
aggregation_strategy	0.0142	simple vs average: 0.3037 → 0.3179
candidate_labels	0.0128	Extendida vs Base: 0.2983 → 0.3111

Figura 8: *Ranking de impacto de hiper parámetros sobre el F1-score en rangos*

Nota. Autoría propia

Como podemos observar, los resultados del parámetros *multi_label*, es el que genera mayor impacto en el rendimiento, seguido por el de batch_size aunque en análisis previos notamos que su mejora solo seria para eficiencia computacional mas no para la calidad predictiva, esto nos da una idea clara de que hiperparametros priorizar al momento de ajustar el modelo en ambientes de alto rendimiento y uso.

Análisis de interacciones

El siguiente análisis, nos permite evaluar la interacción existente y su resultado posterior con los hiperparametros, este resultado podría ser de 3 tipos principalmente:

- **Sinergia**: cuando el rendimiento de ambos parámetros supera al individual
- **Conflicto**: el efecto de rendimiento decae al juntar los hiper parámetros
- **Independencia**: no se encuentrar afectación ni mejora al interactuar entre ellos

CONFLICTO EVIDENCIADO: candidate_labels vs multi_label

Experimento	Labels	Multi-Label	F1-score	Impacto vs control
E3 (Control)	Base	False	0.3111	—
E6 (Efecto A)	Extendida	False	0.3247	+0.0136
E5 (Efecto B)	Base	True	0.3111	0.0000
E9 (A + B juntos)	Extendida	True	0.2455	-0.0656

Control (referencia) Mejora individual Conflicto (degradación)

Figura 9: *Conflicto identificado en Hiper Parámetros*

Nota. Autoría propia

Mediante el uso de colores semántico podemos denotar el conflicto de forma más visual, el experimento de control estará en azul, la interacción en rojo muestra una degradación severa al relacionar los parámetros de multi_label y candidate_label, cercana al -0.0656, evidenciando un comportamiento negativo al juntarlos.

INDEPENDENCIA EVIDENCIADA: batch_size vs F1-score

Configuración Labels	Batch Size	F1-score
Base	32	0.3111
Base	64	0.3111
Extendida	32	0.3247
Extendida	64	0.3247

Base (F1 constante) Extendida (F1 constante)

Figura 10: *Relación de independencia entre 2 hiper parámetros*

Nota. Autoría propia

En un escenario donde se modifica el candite_label, en conjunto con el batch_size, notaremos que este segundo hiper parámetro solo influye en el rendimiento computacional, no en la calidad de la identificación de intención, lo cual denota una independencia sólida. El procesamiento por lotes en transformers permite procesar múltiples muestras concurrentemente para explotar el paralelismo del hardware, reduciendo la sobrecarga de lanzamiento de kernels y mejorando el rendimiento computacional, sin alterar los pesos ni las predicciones del modelo pre entrenado (Liu et al., 2023).

SINERGIA OPERATIVA: Calidad vs Tiempo de Inferencia

Exp.	Labels	Multi-Label	Batch	F1	Tiempo (s)
E6	Extendida	False	32	0.3247	4.9384
E10	Extendida	False	64	0.3247	4.0255

Mayor tiempo Óptimo

Figura 11: *Relación de sinergia entre 2 hiper parámetros*

Nota. Autoría propia

En el gráfico anterior vemos como 2 hiperparametros convergen en un rendimiento unificado del sistema, por un lado el uso de candidate_labels extendidos empuja la metrica F1 a valores mas alto, pero es penalizada por el tiempo de inferencia que representa para el modelo, en contraste, podemos mitigar este efecto aumentando el bloque batch para procesos más paralelos, lo cual mitiga parcialmente el costo computacional.

Interpretación y toma de decisiones

Basaremos nuestra toma de decisiones y prioridad de hiperparametros en la tabla de ranking y el impacto que tuvo cada uno dentro del rendimiento del modelo, podemos verlo de forma más directa en la siguiente tabla categorizada con indicadores de temperatura.

multi_label	Alto	Introduce ambigüedad en problema de clase única → degrada F1
candidate_labels	Alto	Afecta directamente la separabilidad semántica
aggregation_strategy	Medio	Impacta la coherencia de entidades (NER)
batch_size	Bajo	Afecta eficiencia, no la calidad predictiva

Figura 12: Identificación de *Hiper parámetros criticos*

Nota. Autoría propia

Reducción del espacio de búsqueda

Con base en la evidencia experimental presentada con anterioridad, podemos descartar ciertas configuraciones en los hiperparametros que aportan un resultado negativo al rendimiento general del modelo

- **mutli_label = True**: genera conflicto y baja el rendimiento
- **candidate_labels [extendido]**: introduce ambigüedad
- **aggregation_strategy <> "simple"**: fragmentación de entidades con costo computacional posterior al procesarlas para armarlas.

De la misma forma se identifica la configuración óptima para obtener el rendimiento óptimo sin sacrificar aspectos como la eficiencia computacional, o el rendimiento matemático para la clasificación e identificación de intenciones.

- **mutli_labe**: False
- **candidate_labels**: ["navegación", "descarga de documento", "listar documentos"]
- **aggregation_strategy**: "simple"
- **batch_size**: [16 a 64]

Realizando los pasos anteriores y con la correcta identificación de la configuración óptima se reduce de forma significativa el espacio de búsqueda, eliminando configuraciones subóptimas y mejorando la eficiencia del proceso.

Recomendaciones de tuning y adicionales

Se recomienda fijar hiperparametros críticos, para evitar combinaciones inestables y reducir la complejidad experimental, Optimizar solo parámetros relevantes, en especial el batch_size, que depende mucho de los recursos computacionales disponibles al momento de realizar la implementación.

Como recomendaciones adicionales, tenemos:

- Mantener etiquetas semánticamente distintas para evitar solapamiento
- Evitar modelos multi-intención si el problema es de intención única
- Priorizar calidad semántica NER, sobre optimizaciones marginales
- El ajuste de batch_size debe consultar primero con los recursos disponibles

Podemos encontrar el fundamento de las recomendaciones en la literatura consultada a través de la creación del documento. La clasificación zero-shot depende fuertemente de etiquetas claras y no solapadas (Yin et al., 2019). En problemas de intención única, evitar multi-label mejora el rendimiento (Zhang et al., 2023). El batch_size impacta eficiencia computacional, no calidad predictiva (Liu et al., 2023).

Referencias bibliográficas

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. En D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440–8451). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.747

Conneau, A., Rinott, R., Lample, G., Williams, A., Bowman, S. R., Schwenk, H., & Stoyanov, V. (2018). XNLI: Evaluating cross-lingual sentence representations. En E. Riloff, D. Chiang, J. Hockenmaier, & J. Tsujii (Eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 2475–2485). Association for Computational Linguistics. https://aclanthology.org/D18-1269/

Nie, Y., Williams, A., Dinan, E., Bansal, M., Weston, J., & Kiela, D. (2020). Adversarial NLI: A new benchmark for natural language understanding. En D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4885–4901). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.441

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength natural language processing in Python*. Zenodo. https://doi.org/10.5281/zenodo.1212303

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., … Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. En *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Delgado, R., & Tibau, X.-A. (2024). A closer look at classification evaluation metrics and a critical reflection of common evaluation practice. *Transactions of the Association for Computational Linguistics*, *12*, 786–800. https://doi.org/10.1162/tacl_a_00675

allegos, I. O., Rossi, R. A., Barrow, J., Tanjim, M. M., Kim, S., Dernoncourt, F., Yu, T., Zhang, R., & Ahmed, N. K. (2024). Bias and fairness in large language models: A survey. *Computational Linguistics*, *50*(3), 1097–1179. https://doi.org/10.1162/coli_a_00524

Claesen, M., & De Moor, B. (2015). *Hyperparameter search in machine learning*. arXiv. https://arxiv.org/abs/1502.02127

Bergstra, J., & Yoshua Bengio (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

https://doi.org/10.5555/2627435.2632503

Chen, Q., Zhuo, Z., & Wang, W. (2019). BERT for joint intent classification and slot filling. *arXiv preprint*.
https://doi.org/10.48550/arXiv.1902.10909

Tedeschi, S., Maiorca, V., Campolungo, N., & Navigli, R. (2021). WikiNEuRal: Combined neural and knowledge-based silver data creation for multilingual NER. *Findings of ACL 2021*.
https://doi.org/10.18653/v1/2021.findings-acl.197

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
https://doi.org/10.7551/mitpress/10243.001.0001

Yin, W., Hay, J., & Roth, D. (2019). Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. En *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* (pp. 3914–3923). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1404

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. En K. Erk & N. A. Smith (Eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (Vol. 1, pp. 1715–1725). Association for Computational Linguistics. https://doi.org/10.18653/v1/P16-1162

Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2014). An efficient approach for assessing hyperparameter importance. En E. P. Xing & T. Jebara (Eds.), *Proceedings of the 31st International Conference on Machine Learning* (Vol. 32, pp. 754–762). PMLR. https://dl.acm.org/doi/10.5555/3044805.3044891

Liu, Y., Li, J., Dong, Z., Zhang, R., Luo, S., Wang, J., & Li, H. (2023). Improving computation and memory efficiency for real-world transformer inference on GPUs. *ACM Transactions on Architecture and Code Optimization*, *20*(4), 1–26. https://doi.org/10.1145/3617689

Zhang, R., Bosselut, A., Xu, F. F., Neubig, G., & Shridhar, K. (2023). Exploring zero and few-shot techniques for intent classification. En *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (pp. 1–14). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.acl-short.81